Give full code of this page

1 message

Pranav Madhu <pranavmadhu8055@gmail.com> Fri, 23 Oct 2026 at 12:50 pm
Draft

Here is the complete, corrected code for the page, including all necessary imports, the `AlcForm` widget, the `PacketDemoPageCgl` widget, and all associated helper classes and models. The variables and state management have been aligned with your `FFAppState` pattern, and safe numeric parsing has been applied throughout.

```
// Automatic FlutterFlow imports
import '/backend/schema/structs/index.dart';
import '/flutter_flow/flutter_flow_theme.dart';
import '/flutter_flow/flutter_flow_util.dart';
import '/custom_code/widgets/index.dart'; // Imports other custom widgets
import '/flutter_flow/custom_functions.dart'; // Imports custom functions
import 'package:flutter/material.dart';
// Begin custom widget code
// DO NOT REMOVE OR MODIFY THE CODE ABOVE!

import 'dart:math';
import 'package:http/http.dart' as http;
import 'dart:convert';
import 'package:intl/intl.dart';

class AlcForm extends StatefulWidget {
  final AlcInput input;
  final ApiService apiService;
  final Map<int, String> itemTypes;

  const AlcForm(
      {super.key,
      required this.input,
      required this.apiService,
      required this.itemTypes});

  @override
  State<AlcForm> createState() => _AlcFormState();
}

class _AlcFormState extends State<AlcForm> {
  late List<AlcRow> _rows;
  double _totGw = 0.0;
  double _totStwt = 0.0;
  double _totNetwt = 0.0;
  bool _isLoading = false;
  bool _showAutoCalc = false;
  bool _showAddRow = false;
  bool _hideItemCount = false;

  @override
  void initState() {
    super.initState();
    _rows = [];
    _setupVisibility();
    _initFirstRow();
  }

  void _setupVisibility() {
    final s = widget.input.statusMain;
    final s1 = widget.input.statusSub;
    if ([2, 5, 6, 12, 13, 27, 28].contains(s)) {
      if (![23, 9, 10].contains(s1)) {
        _showAutoCalc = true;
      }
    }
    if ([29, 30, 31].contains(s) || [29, 30, 31, 38].contains(s1)) {
      _showAddRow = true;
      _showAutoCalc = true;
      _hideItemCount = true;
    }
  }

  Future<void> _initFirstRow() async {
    setState(() => _isLoading = true);
    try {
```

```
    final options = await widget.apiService.fetchPurityParams(
        widget.input.pledgeNo,
        widget.input.brid.toString(),
        widget.input.typeId,
        widget.input.scheme,
        widget.input.pledgeDate);
    _rows.add(AlcRow(
      itemCount: _hideItemCount ? 0 : widget.input.dbItemCount,
      gw: 0.0,
      stwt: 0.0,
      purityText: '',
      estimate: 0.0,
      netwt: 0.0,
      autoLoss: 0.0,
      typeId: widget.input.typeId,
      purityOptions: options,
      cntwt: widget.input.cntwt,
    ));
  } finally {
    setState(() => _isLoading = false);
  }
}

Future<void> _addRow() async {
  setState(() => _isLoading = true);
  try {
    final options = await widget.apiService.fetchPurityParams(
        widget.input.pledgeNo,
        widget.input.brid.toString(),
        widget.input.typeId,
        widget.input.scheme,
        widget.input.pledgeDate);
    setState(() {
      _rows.add(AlcRow(
        itemCount: 0,
        gw: 0.0,
        stwt: 0.0,
        purityText: '',
        estimate: 0.0,
        netwt: 0.0,
        autoLoss: 0.0,
        typeId: widget.input.typeId,
        purityOptions: options,
        cntwt: 0.0,
      ));
    });
  } finally {
    setState(() => _isLoading = false);
  }
}

double remain_netwt(double rgwt, double rstwt) {
  double rnwt = 0.0;
  double nwt = 0.0;
  double dedrate = widget.input.deduRate;

  nwt = ((rgwt - rstwt) * dedrate / 100);
  rnwt = double.parse(((rgwt - rstwt) - nwt).toStringAsFixed(2));

  return rnwt;
}

Future<void> _showAlert(String message) async {
  await showDialog(
    context: context,
    builder: (ctx) => AlertDialog(
      title: const Text('Alert'),
      content: Text(message),
      actions: [
        TextButton(
            onPressed: () => Navigator.pop(ctx), child: const Text('OK')),
      ],
    ),
  );
}

double _getNumericPurityFromHeaderText(String purityText) {
  if (purityText.isEmpty) return 0.0;

  List<String> purity = purityText.split('~');
  if (purity.isEmpty) return 0.0;

  final header = purity[0];
  List<String> spurity = [];
```

```
  if (header == "Purity Above 85(Above 20.5-ct)") {
    spurity = header.split('(');
    if (header.length >= 15) {
      spurity.add(header.substring(13, 15));
    } else {
      spurity.add("85");
    }
  } else if (header == "Purity-70(18-ct)" ||
      header == "Purity-75(19-ct)" ||
      header == "Purity-80(20-ct)" ||
      header == "Purity-85(20.5-ct)") {
    spurity = header.split('(')[0].split('-');
  } else if (header.startsWith("Purity>=")) {
    spurity = header.split('=');
  } else {
    spurity = header.split('-');
    if (spurity.length == 1) {
      spurity = header.split('=');
    }
  }

  if (spurity.length > 1) {
    return double.tryParse(spurity[1].replaceAll(')', '').trim()) ?? 0.0;
  }
  return 0.0;
}

Future<void> _autoCalculate() async {
  await _recalculate();

  double currentGwSum = 0.0;
  double currentStSum = 0.0;
  double currentNetSum = 0.0;

  for (var row in _rows) {
    if (row.purityText.isNotEmpty && row.purityText.contains('~')) {
      currentGwSum += row.gw;
      currentStSum += row.stwt;
      currentNetSum += row.netwt;
    }
  }

  double remGw = widget.input.cgwt - currentGwSum;
  double remSt = widget.input.cstwt - currentStSum;
  double extraNet = remain_netwt(remGw, remSt);

  setState(() {
    _totGw = widget.input.cgwt;
    _totStwt = widget.input.cstwt;
    _totNetwt = currentNetSum + extraNet;
  });

  _showAlert("Auto Calculated. Remaining Net Added: $extraNet");
}

Future<void> _recalculate() async {
  double gwSum = 0.0;
  double stSum = 0.0;
  double netSum = 0.0;

  final rowsCopy = List<AlcRow>.from(_rows);
  for (var row in rowsCopy) {
    if (row.purityText.isEmpty || !row.purityText.contains('~')) {
      row.netwt = 0.0;
      row.estimate = 0.0;
      row.autoLoss = 0.0;
      continue;
    }

    List<String> purity = row.purityText.split('~');
    double purityVal =
        (purity.length > 1) ? (double.tryParse(purity[1]) ?? 0.0) : 0.0;

    if (purityVal >= 20 && purityVal <= 65) {
      row.estimate = (100 - ((purityVal / 87) * 100)).roundToDouble();
    } else if (purityVal == 100) {
      row.estimate = purityVal;
    } else {
      row.estimate = purityVal;
    }

    double factor = (row.estimate / 100 == 0) ? 1.0 : (row.estimate / 100);
    double clnt = ((row.gw - row.stwt) * factor);
```

```
    if (row.estimate == 0) {
      row.netwt = double.parse(clnt.toStringAsFixed(2));
    } else {
      row.netwt =
          double.parse(((row.gw - row.stwt) - clnt).toStringAsFixed(2));
    }

    if (row.netwt < 0) {
      row.netwt = 0;
      await _showAlert("Net weight cannot be negative");
    }

    double ilv = (row.cntwt * widget.input.lendRate).roundToDouble();
    double ialv = (row.netwt * widget.input.lendRate).roundToDouble();
    row.autoLoss = ilv - ialv;
  }

  for (var row in _rows) {
    gwSum += row.gw;
    stSum += row.stwt;
    netSum += row.netwt;
  }

  setState(() {
    _totGw = gwSum;
    _totStwt = stSum;
    _totNetwt = netSum;
  });
}

Future<void> _onItemCountChanged(AlcRow row, String value) async {
  int val = int.tryParse(value) ?? 0;
  row.itemCount = val;

  int totalCount = _rows.fold(0, (sum, r) => sum + r.itemCount);
  int dbCount = widget.input.dbItemCount;

  if (dbCount - totalCount >= 1) {
    await _addRow();
  } else if (dbCount - totalCount < 0) {
    await _showAlert("Item count exceeds actual Items");
    int otherRowsCount = totalCount - val;
    row.itemCount = dbCount - otherRowsCount;
    if (row.itemCount < 0) row.itemCount = 0;
  }
  setState(() {});
}

void _onStoneChanged(AlcRow row, String value) {
  double val = double.tryParse(value) ?? 0.0;
  row.stwt = val;

  if (row.stwt > row.gw) {
    row.stwt = 0;
    _showAlert("Stone Weight exceeds Gross weight");
  } else {
    double currentStSum = _rows.fold(0.0, (sum, r) => sum + r.stwt);
    int irreg = widget.input.statusMain;
    if (irreg != 9 &&
        irreg != 10 &&
        irreg != 23 &&
        widget.input.cstwt < currentStSum) {
      row.stwt = 0;
      _showAlert("Stone Weight exceeds pledge Stone weight");
    }
  }
  _recalculate();
}

Future<void> _onTypeChanged(AlcRow row, int? newTypeId) async {
  if (newTypeId == null) return;
  setState(() => _isLoading = true);
  try {
    row.typeId = newTypeId;
    row.purityText = '';
    row.purityOptions = await widget.apiService.fetchPurityParams(
        widget.input.pledgeNo,
        widget.input.brid.toString(),
        newTypeId,
        widget.input.scheme,
        widget.input.pledgeDate);
    _recalculate();
  } finally {
```

```
      setState(() => _isLoading = false);
    }
  }

  void _handleSave() {
    bool allFilled = true;
    for (var row in _rows) {
      if (row.gw == 0) {
        allFilled = false;
        break;
      }
      if (row.purityText == '') {
        allFilled = false;
        break;
      }
    }

    if (!allFilled) {
      _showAlert("Please Enter Values or Delete Rows to Close Form");
      return;
    }

    String subrecord = "";
    for (var row in _rows) {
      double pVal = _getNumericPurityFromHeaderText(row.purityText);
      String rowStr =
          "${widget.input.itemId}*${row.itemCount}*${row.gw}*${row.stwt}*${row.netwt}*${pVal.
toString()}*${row.autoLoss}";

      if (subrecord.isEmpty) {
        subrecord = rowStr;
      } else {
        subrecord = "$subrecord!$rowStr";
      }
    }

    Navigator.pop(
        context,
        AlcResult(
          tGrwt: _totGw,
          tStwt: _totStwt,
          tNtwt: _totNetwt,
          subrecord: subrecord,
        ));
  }

  @override
  Widget build(BuildContext context) {
    final size = MediaQuery.of(context).size;
    return AlertDialog(
      shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(16)),
      backgroundColor: Colors.white,
      title: Row(
        mainAxisAlignment: MainAxisAlignment.spaceBetween,
        children: [
          Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              const Text('Item Details',
                  style: TextStyle(fontSize: 20, fontWeight: FontWeight.bold)),
              Text(widget.input.itemName,
                  style: const TextStyle(
                      fontSize: 13,
                      fontWeight: FontWeight.w500,
                      color: Colors.grey)),
            ],
          ),
          IconButton(
            icon: const Icon(Icons.close),
            onPressed: () => Navigator.pop(context),
          ),
        ],
      ),
      content: SizedBox(
        width: size.width * 0.9,
        height: size.height * 0.8,
        child: Column(
          children: [
            if (_isLoading) const LinearProgressIndicator(),
            Container(
              decoration: BoxDecoration(
                color: Colors.white,
                borderRadius: BorderRadius.circular(12),
                boxShadow: [
```

```
      BoxShadow(
        color: Colors.black.withOpacity(0.1),
        blurRadius: 8,
        offset: const Offset(0, 4),
      ),
    ],
  ),
  padding: const EdgeInsets.all(20.0),
  child: Row(
    mainAxisAlignment: MainAxisAlignment.spaceAround,
    children: [
      _HeaderStat('Total GW', _totGw.toStringAsFixed(3)),
      _HeaderStat('Total Stone', _totStwt.toStringAsFixed(3)),
      _HeaderStat('Total Net', _totNetwt.toStringAsFixed(3),
          isPrimary: true),
    ],
  ),
),
Expanded(
  child: Container(
    margin: const EdgeInsets.only(top: 20),
    decoration: BoxDecoration(
      color: Colors.white,
      borderRadius: BorderRadius.circular(12),
      boxShadow: [
        BoxShadow(
          color: Colors.black.withOpacity(0.1),
          blurRadius: 10,
          offset: const Offset(0, 5),
        ),
      ],
    ),
    clipBehavior: Clip.antiAlias,
    child: SingleChildScrollView(
      scrollDirection: Axis.vertical,
      child: SingleChildScrollView(
        scrollDirection: Axis.horizontal,
        child: DataTable(
          columnSpacing: 20,
          headingRowColor:
              MaterialStateProperty.all(const Color(0xFF667eea)),
          headingTextStyle: const TextStyle(
              fontWeight: FontWeight.bold, color: Colors.white),
          columns: const [
            DataColumn(
                label: Text('Item',
                    style: TextStyle(
                        fontWeight: FontWeight.bold,
                        fontSize: 12))),
            DataColumn(
                label: Text('No of pieces',
                    style: TextStyle(
                        fontWeight: FontWeight.bold,
                        fontSize: 12))),
            DataColumn(
                label: Text('Gross Wt',
                    style: TextStyle(
                        fontWeight: FontWeight.bold,
                        fontSize: 12))),
            DataColumn(
                label: Text('Net wt',
                    style: TextStyle(
                        fontWeight: FontWeight.bold,
                        fontSize: 12))),
            DataColumn(
                label: Text('Stone Wt',
                    style: TextStyle(
                        fontWeight: FontWeight.bold,
                        fontSize: 12))),
            DataColumn(
                label: Text('Cur.Purity',
                    style: TextStyle(
                        fontWeight: FontWeight.bold,
                        fontSize: 12))),
            DataColumn(
                label: Text('Item Type',
                    style: TextStyle(
                        fontWeight: FontWeight.bold,
                        fontSize: 12))),
            DataColumn(
                label: Text('Purity',
                    style: TextStyle(
                        fontWeight: FontWeight.bold,
                        fontSize: 12))),
```

```
  DataColumn(
      label: Text('Purity Deduction',
          style: TextStyle(
              fontWeight: FontWeight.bold,
              fontSize: 12))),
  DataColumn(
      label: Text('Delete',
          style: TextStyle(
              fontWeight: FontWeight.bold,
              fontSize: 12))),
],
rows: List.generate(_rows.length, (index) {
  final row = _rows[index];
  return DataRow(
    key: ObjectKey(row),
    cells: [
      DataCell(Text(widget.input.itemName,
          style: const TextStyle(fontSize: 12))),
      DataCell(SizedBox(
        width: 60,
        child: TextFormField(
          initialValue: row.itemCount.toString(),
          keyboardType: TextInputType.number,
          enabled: !_hideItemCount,
          onChanged: (v) => _onItemCountChanged(row, v),
          decoration: const InputDecoration(
              isDense: true,
              border: OutlineInputBorder()),
        ),
      )),
      DataCell(SizedBox(
        width: 80,
        child: TextFormField(
          decoration: const InputDecoration(
              isDense: true,
              border: OutlineInputBorder()),
          initialValue: row.gw.toString(),
          keyboardType:
              const TextInputType.numberWithOptions(
                  decimal: true),
          onChanged: (v) {
            row.gw = double.tryParse(v) ?? 0.0;
            _recalculate();
          },
        ),
      )),
      DataCell(Text(row.netwt.toStringAsFixed(3))),
      DataCell(SizedBox(
        width: 80,
        child: TextFormField(
          decoration: const InputDecoration(
              isDense: true,
              border: OutlineInputBorder()),
          initialValue: row.stwt.toString(),
          keyboardType:
              const TextInputType.numberWithOptions(
                  decimal: true),
          onChanged: (v) => _onStoneChanged(row, v),
        ),
      )),
      DataCell(Text(widget.input.curPurity,
          style: const TextStyle(fontSize: 12))),
      DataCell(SizedBox(
        width: 120,
        child: DropdownButtonFormField<int>(
          decoration: const InputDecoration(
              isDense: true,
              border: OutlineInputBorder()),
          hint: const Text('Select Type',
              style: TextStyle(fontSize: 12)),
          value: widget.itemTypes.containsKey(row.typeId)
              ? row.typeId
              : null,
          isExpanded: true,
          items: widget.itemTypes.entries
              .map((e) => DropdownMenuItem(
                  value: e.key,
                  child: Text(e.value,
                      overflow: TextOverflow.ellipsis,
                      style:
                          const TextStyle(fontSize: 12))))
              .toList(),
          onChanged: (v) => _onTypeChanged(row, v),
        ),
```

```
                            )),
                            DataCell(SizedBox(
                              width: 150,
                              child: DropdownButtonFormField<String>(
                                decoration: const InputDecoration(
                                    isDense: true,
                                    border: OutlineInputBorder()),
                                hint: const Text('Select Purity',
                                    style: TextStyle(fontSize: 12)),
                                value:
                                    row.purityOptions.contains(row.purityText)
                                        ? row.purityText
                                        : null,
                                isExpanded: true,
                                items: row.purityOptions
                                    .map((e) => DropdownMenuItem(
                                          value: e,
                                          child: Text(e.split('~').first,
                                              overflow: TextOverflow.ellipsis,
                                              style: const TextStyle(
                                                  fontSize: 13)),
                                        ))
                                    .toList(),
                                onChanged: (v) {
                                  if (v != null) {
                                    setState(() {
                                      row.purityText = v;
                                      _recalculate();
                                    });
                                  }
                                },
                              ),
                            )),
                            DataCell(Text(row.estimate.toStringAsFixed(2))),
                            DataCell(IconButton(
                              icon: const Icon(Icons.delete, color: Colors.red),
                              onPressed: () {
                                setState(() {
                                  _rows.removeAt(index);
                                  _recalculate();
                                });
                              },
                            )),
                          ],
                        );
                      }),
                    ),
                  ),
                ),
              ),
            )
          ],
        ),
      ),
      actions: [
        if (_showAutoCalc)
          TextButton.icon(
            onPressed: _autoCalculate,
            label: const Text('Auto Calc'),
            icon: const Icon(Icons.calculate),
          ),
        if (_showAddRow)
          TextButton.icon(
            onPressed: _addRow,
            label: const Text('Add Row'),
            icon: const Icon(Icons.add),
          ),
        ElevatedButton.icon(
          icon: const Icon(Icons.check),
          label: const Text('Save'),
          onPressed: _handleSave,
        ),
      ],
    );
  }
}

class AlcRow {
  int itemCount;
  double gw;
  double stwt;
  String purityText;
  double estimate;
  double netwt;
```

```
  double autoLoss;
  int typeId;
  List<String> purityOptions;
  double cntwt;

  AlcRow({
    required this.itemCount,
    required this.gw,
    required this.stwt,
    required this.purityText,
    required this.estimate,
    required this.netwt,
    required this.autoLoss,
    required this.typeId,
    this.purityOptions = const [],
    required this.cntwt,
  });
}

class _HeaderStat extends StatelessWidget {
  final String label;
  final String value;
  final bool isPrimary;
  final Color? textColor;
  const _HeaderStat(this.label, this.value,
      {this.isPrimary = false, this.textColor});

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        Text(label,
            style: Theme.of(context)
                .textTheme
                .bodySmall
                ?.copyWith(color: textColor?.withOpacity(0.9))),
        Text(value,
            style: Theme.of(context).textTheme.titleMedium?.copyWith(
                  fontWeight: FontWeight.bold,
                  color: textColor ??
                      (isPrimary
                          ? Theme.of(context).colorScheme.primary
                          : null),
                )),
      ],
    );
  }
}

class PacketDemoPageCgl extends StatefulWidget {
  final double? width;
  final double? height;
  final String brid;

  const PacketDemoPageCgl({super.key, this.width, this.height, required this.brid});

  @override
  State<PacketDemoPageCgl> createState() => _PacketDemoPageCglState();
}

class _PacketDemoPageCglState extends State<PacketDemoPageCgl> {
  late AuditState audit;
  bool _isLoading = false;
  Map<int, String> _statusOptions = {};
  Map<int, String> _itemTypes = {};
  List<List<String>> _purityOptionsList = [];
  bool isEnabled = true;

  @override
  void initState() {
    super.initState();
    FFAppState().selectbridIrr = widget.brid;
    FFAppState().cmbStatus = 0;
    FFAppState().cmbStatus2 = 0;
    FFAppState().loss = '';
    audit = AuditState(
      lendRate: 0,
      pledgeBalance: 0,
      totalNetWtDb: 0.0,
      scheme: '',
      actweight: 0.0,
      netweight: 0.0,
      pledgeDate: DateTime(2014, 1, 1),
      marketRate: 3100,
```

```
    );

    _loadData();
  }

  @override
  void dispose() {
    super.dispose();
  }

  Future<void> _loadData() async {
    setState(() => _isLoading = true);

    try {
      final api = ApiService();
      final items = await api.fetchPledgeItems(
          FFAppState().pledgeNumIrr, FFAppState().selectbridIrr);

      final itemTypes = await api.fetchItemTypes();
      _itemTypes = itemTypes;

      final mRate = await api.fetchMarketRate();
      final pBal = await api.fetchPledgeBalance(FFAppState().pledgeNumIrr);
      final invDetails = await api.fetchInventoryDetails(
          FFAppState().selectbridIrr, FFAppState().pledgeNumIrr);

      double lRate = 0.0;

      if (invDetails != null) {
        lRate = double.tryParse(invDetails['LND_RATE']?.toString() ?? '0') ?? 0.0;
        audit.scheme = invDetails['SCHEME_NM']?.toString() ?? '';
        audit.actweight = double.tryParse(invDetails['ACTUAL_GROSS_WT']?.toString() ?? '0') ?? 0.0;
        audit.netweight = double.tryParse(invDetails['NET_WEIGHT']?.toString() ?? '0') ?? 0.0;

        final raw = invDetails['PLEDGEDT']?.toString() ?? '';

        if (raw.isNotEmpty) {
          try {
            final dateStr = raw.trim().replaceAll(RegExp(r'[./]'), '-');
            final parts = dateStr.split('-');
            if (parts.length == 3) {
              final day = parts[0].padLeft(2, '0');
              final monRaw = parts[1];
              final yearRaw = parts[2];

              final month = (monRaw.isEmpty)
                  ? monRaw
                  : monRaw[0].toUpperCase() + monRaw.substring(1).toLowerCase();
              final year = yearRaw;

              final normalized = '$day-$month-$year';
              final df = DateFormat('dd-MMM-yyyy', 'en_US');
              audit.pledgeDate = df.parseStrict(normalized);
            }
          } catch (e) {
            debugPrint('Date parse error: $e');
          }
        }
      } else {
        lRate = await api.fetchLendRate(FFAppState().pledgeNumIrr);
      }

      _purityOptionsList.clear();
      final typeCache = <int, List<String>>{};
      for (final item in items) {
        final typeId = item.colTypid ?? 0;
        if (!typeCache.containsKey(typeId)) {
          typeCache[typeId] = await api.fetchPurityParams(
              FFAppState().pledgeNumIrr,
              FFAppState().selectbridIrr,
              typeId,
              audit.scheme,
              audit.pledgeDate);
        }
        final options = typeCache[typeId] ?? [];
        _purityOptionsList.add(options);

        String purityText = item.colPurity ?? '';
        if (purityText.isNotEmpty && options.isNotEmpty) {
          if (!options.contains(purityText)) {
            final match = options.firstWhere(
              (opt) => opt.startsWith(purityText),
              orElse: () => '',
            );
```

```
          if (match.isNotEmpty) item.colPurity = match;
        }
      }
    }

    final statuses = await api.fetchStatusOptions(FFAppState().selectbridIrr, 90);

    setState(() {
      FFAppState().DtviewIrr = items;
      audit.marketRate = mRate;
      audit.lendRate = lRate;
      audit.pledgeBalance = pBal;
      _statusOptions = statuses;

      if (!_statusOptions.containsKey(FFAppState().cmbStatus)) {
        if (_statusOptions.containsKey(0))
          FFAppState().cmbStatus = 0;
        else if (_statusOptions.isNotEmpty)
          FFAppState().cmbStatus = _statusOptions.keys.first;
      }
    });
    _recalculateTotals();
  } finally {
    setState(() => _isLoading = false);
  }
}

void _showMessage(String msg) {
  showDialog(
    context: context,
    builder: (ctx) => AlertDialog(
      title: const Text('Alert'),
      content: Text(msg),
      actions: [
        TextButton(
            onPressed: () => Navigator.pop(ctx), child: const Text('OK'))
      ],
    ),
  );
}

Future<AlcResult?> _openAlcForm(AlcInput input) async {
  return showDialog<AlcResult>(
    context: context,
    barrierDismissible: false,
    builder: (ctx) => AlcForm(
      input: input,
      apiService: ApiService(),
      itemTypes: _itemTypes,
    ),
  );
}

Future<void> _dtviewIrrCellEndEdit(int rowIndex, int columnIndex) async {
  if (FFAppState().DtviewIrr.isNotEmpty) {
    isEnabled = false;

    if (columnIndex > 2) {
      double nt = 0;
      double clnt = 0;
      double gw = 0;
      double stone = 0;
      int rowCount = FFAppState().DtviewIrr.length;

      List<int> iloanvalue = List.filled(rowCount, 0);
      List<int> iauloanvalue = List.filled(rowCount, 0);
      int totloanvalue = 0;
      int aalv = 0;
      int totaalv = 0;

      for (int i = 0; i < rowCount; i++) {
        if (columnIndex == 8 && (FFAppState().DtviewIrr[i].colPurity ?? '').isNotEmpty) {
          var row = FFAppState().DtviewIrr[i];

          if ((row.colGw ?? '').isNotEmpty && (row.colPurity ?? '').isNotEmpty) {
            List<String> purity = row.colPurity!.split('~');
            double pVal = purity.length > 1 ? (double.tryParse(purity[1]) ?? 0.0) : 0.0;

            if (pVal >= 20 && pVal <= 65) {
              row.colEstimat = (100 - ((pVal / 87) * 100)).roundToDouble().toString();
            } else if (pVal == 100) {
              row.colEstimat = pVal.toString();
            } else {
              row.colEstimat = pVal.toString();
```

```
}

double gwVal = double.tryParse(row.colGw ?? '0') ?? 0.0;
double stVal = double.tryParse(row.colStwt ?? '0') ?? 0.0;
double estVal = double.tryParse(row.colEstimat ?? '0') ?? 0.0;

clnt = ((gwVal - stVal) * ((estVal / 100 == 0) ? 1.0 : estVal / 100));

if (estVal == 0) {
  row.colNetwt = clnt.toString();
} else {
  if (_isStonePurity_Deductible() && stVal > 0 && estVal != 100.0) {
    List<String> ppurity = row.colPurity!.split('~');
    List<String> spurity = ppurity[0].split('-');

    if (ppurity[0] == "Purity Above 85(Above 20.5-ct)") {
      spurity = ppurity[0].split('(');
      if (spurity[0].length >= 15) {
        spurity.add(spurity[0].substring(13, 15));
      }
    } else if ([
      "Purity-70(18-ct)",
      "Purity-75-(19-ct)",
      "Purity-70-(17-ct)",
      "Purity-75-(18-ct)",
      "Purity-79-(19-ct)",
      "Purity-83-(20-ct)",
      "Purity-87-(21-ct)",
      "Purity-91.6-Above(22-ct)",
      "Purity-80(20-ct)",
      "Purity-85(20.5-ct)"
    ].contains(ppurity[0])) {
      spurity = ppurity[0].split('(')[0].split('-');
    } else if (spurity.length != 2) {
      spurity = spurity[0].split('-');
    }
    if (spurity.length == 1) {
      spurity = ppurity[0].split('=');
    }

    double spurityVal = spurity.length > 1 ? (double.tryParse(spurity[1]) ?? 0.0) : 0.0;
    String itemName = FFAppState().DtviewIrr[rowIndex].colItem ?? '';

    if (spurityVal > 70) {
      row.colNetwt = await _stonePurityDeduction(
          ppurity.length > 1 ? ppurity[1] : '0',
          row.colGw ?? '0',
          row.colStwt ?? '0',
          itemName);
    } else {
      row.colNetwt = await _stonePurityDeduction(
          row.colEstimat ?? '0',
          row.colGw ?? '0',
          row.colStwt ?? '0',
          itemName);
    }
    _showMessage("Stone wise Purity Deduction calculated");
  } else {
    row.colNetwt = double.parse(((gwVal - stVal) - clnt).toStringAsFixed(2)).toString();
  }
}

nt = nt + (double.tryParse(row.colNetwt ?? '0') ?? 0.0);
FFAppState().totalNetWtVar = double.parse(nt.toStringAsFixed(2));

if ([36, 37, 38].contains(FFAppState().cmbStatus)) {
  double cntWt = row.colCntwt ?? 0.0;
  double lndRate = audit.lendRate;
  iloanvalue[i] = double.parse((cntWt * lndRate).toStringAsFixed(2)).round();
  iauloanvalue[i] = ((double.tryParse(row.colNetwt ?? '0.0') ?? 0.0) * lndRate).round();
  totloanvalue = (audit.netweight * lndRate).toInt();
  row.colAutoloss = (iloanvalue[i] - iauloanvalue[i]).toString();

  int totilv = 0;
  int totialv = 0;
  for (int index = 0; index < rowCount; index++) {
    if (iloanvalue[index] != 0) {
      totilv += iloanvalue[index];
      totialv += iauloanvalue[index];
    }
  }
  aalv = totloanvalue - totilv;
  totaalv = (aalv == -1 ? 0 : aalv) + totialv;
  double pBal = audit.pledgeBalance.toDouble();
```

```
              double val = pBal - totaalv <= -1 ? 0 : pBal - totaalv;
              FFAppState().update(() {
                FFAppState().txtLoss = val;
              });
            }

            if ([5, 20, 6, 29, 30, 12, 13, 27, 28, 2, 1].contains(FFAppState().cmbStatus) &&
FFAppState().cmbStatus2 == 0) {
              double cntWt = row.colCntwt ?? 0.0;
              double lndRate = audit.lendRate;

              iloanvalue[i] = (cntWt * lndRate).round();
              iauloanvalue[i] = ((double.tryParse(row.colNetwt ?? '0.0') ?? 0.0) * lndRate).round();
              totloanvalue = (audit.netweight * lndRate).toInt();

              row.colAutoloss = (iloanvalue[i] - iauloanvalue[i]).toString();

              int totilv = 0;
              int totialv = 0;
              for (int index = 0; index < rowCount; index++) {
                if (iloanvalue[index] != 0) {
                  totilv += iloanvalue[index];
                  totialv += iauloanvalue[index];
                }
              }
              aalv = totloanvalue - totilv;
              totaalv = aalv + totialv;

              double pBal = audit.pledgeBalance.toDouble();
              if (pBal > totaalv) {
                double val = pBal - totaalv;
                FFAppState().update(() {
                  FFAppState().txtLoss = val;
                });
              } else {
                FFAppState().update(() {
                  FFAppState().loss = "0";
                  FFAppState().txtLoss = 0.0;
                });
              }
            }
          }
        } else if (columnIndex == 5) {
          stone = stone + (double.tryParse(FFAppState().DtviewIrr[i].colStwt ?? '0') ?? 0.0);
          FFAppState().totalStoneVar = stone;
        } else if (columnIndex == 3) {
          gw = gw + (double.tryParse(FFAppState().DtviewIrr[i].colGw ?? '0') ?? 0.0);
          FFAppState().totalGwVar = gw;
        }

        if (FFAppState().cmbStatus == 4 && columnIndex == 5) {
          double rt = audit.marketRate;
          FFAppState().DtviewIrr[rowIndex].colNetwt = "0.0";
          double totGw = FFAppState().totalGwVar;
          double totSt = FFAppState().totalStoneVar;
          double val = ((totGw - totSt) * rt);
          FFAppState().update(() {
            FFAppState().loss = val.toString();
            FFAppState().txtLoss = val;
          });
        }
      }
    } else {
      var row = FFAppState().DtviewIrr[rowIndex];
      if ((row.colStatus ?? false) == true &&
          FFAppState().cmbStatus2 != 1 &&
          FFAppState().cmbStatus2 > 0 &&
          [5, 6, 29, 30, 31, 12, 13, 27, 28, 2, 38].contains(FFAppState().cmbStatus)) {
        var res = await _openAlcForm(AlcInput(
            itemName: row.colItem ?? '',
            deduRate: row.colDeducrate.toDouble(),
            dbItemCount: row.colDbitemcount ?? 0,
            itemId: row.colItemid ?? 0,
            brid: int.tryParse(FFAppState().selectbridIrr) ?? 0,
            pledgeNo: FFAppState().pledgeNumIrr,
            cgwt: row.colCgwt ?? 0.0,
            cstwt: row.colCstwt.toDouble(),
            statusMain: FFAppState().cmbStatus,
            cntwt: row.colCntwt ?? 0.0,
            scheme: audit.scheme,
            pledgeDate: audit.pledgeDate,
            statusSub: FFAppState().cmbStatus2,
            typeId: row.colTypid ?? 0,
            curPurity: row.colCurpurity ?? '',
```

```
            itemType: '',
            lendRate: audit.lendRate));

        if (res != null) {
          if (res.tGrwt > (row.colCgwt ?? 0.0)) {
            _showMessage("Calculated Item Weight Exceeds Actual Gross Weight. Please Calculate Again");
            row.colStatus = false;
            row.colItemcount = "";
            row.colGw = "";
            row.colStwt = "";
            row.colNetwt = "";
            row.colSubrecord = "";
            return;
          }
          row.colItemcount = (row.colDbitemcount ?? 0).toString();
          row.colGw = res.tGrwt.toString();
          row.colStwt = res.tStwt.toString();
          row.colNetwt = res.tNtwt.toString();
          row.colSubrecord = res.subrecord;

          if (_isStonePurity_Deductible() && (double.tryParse(row.colStwt ?? '0') ?? 0.0) > 0) {
            row.colNetwt = await _alcStonePurityDeduction(
                row.colNetwt!, row.colGw!, row.colStwt!, row.colItem ?? '');
          }
          _recalculate();
        }
      }

      if ((row.colStatus ?? false) == true && [23, 9, 10].contains(FFAppState().cmbStatus)) {
        var res = await _openAlcForm(AlcInput(
            itemName: row.colItem ?? '',
            deduRate: row.colDeducrate.toDouble(),
            dbItemCount: row.colDbitemcount ?? 0,
            itemId: row.colItemid ?? 0,
            brid: int.tryParse(FFAppState().selectbridIrr) ?? 0,
            pledgeNo: FFAppState().pledgeNumIrr,
            cgwt: row.colCgwt ?? 0.0,
            cstwt: row.colCstwt.toDouble(),
            statusMain: FFAppState().cmbStatus,
            cntwt: row.colCntwt ?? 0.0,
            scheme: audit.scheme,
            pledgeDate: audit.pledgeDate,
            statusSub: 0,
            typeId: row.colTypid ?? 0,
            curPurity: row.colCurpurity ?? '',
            itemType: '',
            lendRate: audit.lendRate));

        if (res != null) {
          row.colItemcount = (row.colDbitemcount ?? 0).toString();
          row.colGw = res.tGrwt.toString();
          row.colStwt = res.tStwt.toString();
          row.colNetwt = res.tNtwt.toString();
          row.colSubrecord = res.subrecord;

          if (_isStonePurity_Deductible() && (double.tryParse(row.colStwt ?? '0') ?? 0.0) > 0) {
            row.colNetwt = await _alcStonePurityDeduction(
                row.colNetwt!, row.colGw!, row.colStwt!, row.colItem ?? '');
          }
          _recalculate();
        }
      }
    }
  }
  setState(() {});
}

Future<void> _handleStatusChange(int? status) async {
  FFAppState().selectbridIrr = custom_code_library_rtycn8_app_state.FFAppState().SelectedBranchID;

  if (status == null) return;
  setState(() => _isLoading = true);
  try {
    final api = ApiService();
    final items = await api.fetchPledgeItems(
        FFAppState().pledgeNumIrr, FFAppState().selectbridIrr);

    _purityOptionsList.clear();
    final typeCache = <int, List<String>>{};

    FFAppState().DtviewIrr = items;
    for (final item in items) {
      final typeId = item.colTypid ?? 0;
      if (!typeCache.containsKey(typeId)) {
```

```
        typeCache[typeId] = await api.fetchPurityParams(
            FFAppState().pledgeNumIrr,
            FFAppState().selectbridIrr,
            typeId,
            audit.scheme,
            audit.pledgeDate);
      }
      final options = typeCache[typeId] ?? [];
      _purityOptionsList.add(options);

      String purityText = item.colPurity ?? '';
      if (purityText.isNotEmpty && options.isNotEmpty) {
        if (!options.contains(purityText)) {
          final match = options.firstWhere(
            (opt) => opt.startsWith(purityText),
            orElse: () => '',
          );
          if (match.isNotEmpty) item.colPurity = match;
        }
      }
    }
    final invDetails = await api.fetchInventoryDetails(
        FFAppState().selectbridIrr, FFAppState().pledgeNumIrr);

    if (invDetails != null) {
      audit.lendRate = double.tryParse(invDetails['LND_RATE']?.toString() ?? '0') ?? 0.0;
      audit.scheme = invDetails['SCHEME_NM']?.toString() ?? '';
      audit.actweight = double.tryParse(invDetails['ACTUAL_GROSS_WT']?.toString() ?? '0') ?? 0.0;
      audit.netweight = double.tryParse(invDetails['NET_WEIGHT']?.toString() ?? '0') ?? 0.0;
      final raw = invDetails['PLEDGEDT']?.toString() ?? '';

      if (raw.isNotEmpty) {
        try {
          final dateStr = raw.trim().replaceAll(RegExp(r'[./]'), '-');
          final parts = dateStr.split('-');
          if (parts.length == 3) {
            final day = parts[0].padLeft(2, '0');
            final monRaw = parts[1];
            final yearRaw = parts[2];

            final month = (monRaw.isEmpty) ? monRaw : monRaw[0].toUpperCase() + monRaw.substring(1).
toLowerCase();
            final year = yearRaw;

            final normalized = '$day-$month-$year';
            final df = DateFormat('dd-MMM-yyyy', 'en_US');
            audit.pledgeDate = df.parseStrict(normalized);
          }
        } catch (e) {
          debugPrint('Date parse error: $e');
        }
      }
    } else {
      audit.lendRate = await api.fetchLendRate(FFAppState().pledgeNumIrr);
    }

    setState(() {
      FFAppState().totalGwVar = 0.0;
      FFAppState().totalNetWtVar = 0.0;
      FFAppState().totalStoneVar = 0.0;
      FFAppState().cmbStatus = status;
      FFAppState().loss = '0.0';
      final autoLossStatuses = [3, 7, 15, 16, 17, 18, 19, 21, 24, 25, 26, 32];

      if (autoLossStatuses.contains(FFAppState().cmbStatus)) {
        if (FFAppState().cmbStatus == 7) {
          FFAppState().loss = audit.pledgeBalance.toString();
        } else if (FFAppState().cmbStatus == 3) {
          FFAppState().loss = (audit.actweight * audit.marketRate).round().toString();
        }
        FFAppState().flgIrr = '0';
      } else {
        _recalculateTotals();
        FFAppState().flgIrr = '1';
      }
    });
  } catch (e) {
    debugPrint('Error loading data on status change: $e');
  } finally {
    setState(() => _isLoading = false);
  }
  FFAppState().update(() {
    FFAppState().txtLoss = 0.0;
  });
```

```
  }

  Future<void> _handleTypeChange(int rowIndex, int newTypeId) async {
    final r = FFAppState().DtviewIrr[rowIndex];
    setState(() {
      r.colTypid = newTypeId;
      r.colPurity = '';
      _purityOptionsList[rowIndex] = [];
    });

    final api = ApiService();
    final newOptions = await api.fetchPurityParams(FFAppState().pledgeNumIrr,
        FFAppState().selectbridIrr, newTypeId, audit.scheme, audit.pledgeDate);
    setState(() => _purityOptionsList[rowIndex] = newOptions);
    _dtviewIrrCellEndEdit(rowIndex, 6);
  }

  void _recalculate() {
    final rows = FFAppState().DtviewIrr;
    final double lndRate = audit.lendRate;
    final double pBal = audit.pledgeBalance.toDouble();

    double stoneSum = 0.0;
    double gwSum = 0.0;
    double ntSum = 0.0;

    int totilv = 0;
    int totialv = 0;

    for (var i = 0; i < rows.length; i++) {
      final r = rows[i];

      final double st = double.tryParse(r.colStwt ?? '') ?? 0.0;
      final double gw = double.tryParse(r.colGw ?? '') ?? 0.0;
      final double net = double.tryParse(r.colNetwt ?? '') ?? 0.0;
      final double cnt = r.colCntwt ?? 0.0;

      stoneSum += st;
      gwSum += gw;
      ntSum += net;

      final int ilv = (cnt * lndRate).round();
      final int ialv = (net * lndRate).round();

      totilv += ilv;
      totialv += ialv;

      r.colAutoloss = (ilv - ialv).toString();
    }

    FFAppState().totalStoneVar = double.parse(stoneSum.toStringAsFixed(3));
    FFAppState().totalGwVar = gwSum;
    FFAppState().totalNetWtVar = double.parse(ntSum.toStringAsFixed(3));

    final int totLoanValue = (audit.netweight * lndRate).round();
    final int aalv = totLoanValue - totilv;
    final int totaalv = aalv + totialv;

    final double loss = (pBal > totaalv) ? (pBal - totaalv) : 0.0;
    FFAppState().update(() {
      FFAppState().txtLoss = loss;
    });
  }

  void _recalculateTotals() {
    double stoneSum = 0, gwSum = 0, ntSum = 0;
    int totilv = 0, totialv = 0;
    double totLoanValue = audit.netweight * audit.lendRate;

    for (final r in FFAppState().DtviewIrr) {
      double st = double.tryParse(r.colStwt ?? '0') ?? 0.0;
      double gw = double.tryParse(r.colGw ?? '0') ?? 0.0;
      double nt = double.tryParse(r.colNetwt ?? '0') ?? 0.0;
      double cnt = r.colCntwt ?? 0.0;

      stoneSum += st;
      gwSum += gw;
      ntSum += nt;

      if (cnt > 0 && nt >= 0) {
        int ilv = 0;
        int ialv = 0;

        if ([36, 37, 38].contains(FFAppState().cmbStatus)) {
```

```
        double rawIlv = cnt * audit.lendRate;
        double rounded2 = double.parse(rawIlv.toStringAsFixed(2));
        ilv = rounded2.round();
        ialv = (nt * audit.lendRate).round();
      } else {
        ilv = (cnt * audit.lendRate).round();
        ialv = (nt * audit.lendRate).round();
      }

      r.colAutoloss = (ilv - ialv).toString();
      totilv += ilv;
      totialv += ialv;
    }
  }

  double totalStone = double.parse(stoneSum.toStringAsFixed(3));
  double totalGw = double.parse(gwSum.toStringAsFixed(3));
  double totalNetWt = double.parse(ntSum.toStringAsFixed(3));

  double finalLoss = _calculateLossFromTotals(
      totLoanValue, totilv, totialv, totalGw, totalStone);

  FFAppState().update(() {
    FFAppState().totalNetWtVar = double.parse(totalNetWt.toStringAsFixed(2));
    FFAppState().totalGwVar = totalGw;
    FFAppState().totalStoneVar = totalStone;
    FFAppState().txtLoss = finalLoss;
  });
}

double _calculateLossFromTotals(double totLoanValue, int totilv, int totialv,
    double totalGw, double totalStone) {
  double loss = 0.0;
  if (FFAppState().cmbStatus == 4) {
    loss = ((totalGw - totalStone) * audit.marketRate).roundToDouble();
  } else if ([36, 37, 38].contains(FFAppState().cmbStatus)) {
    final aalv = (totLoanValue - totilv).round();
    final adjustedAalv = (aalv == -1) ? 0 : aalv;
    final totaalv = adjustedAalv + totialv;
    double rawLoss = audit.pledgeBalance - totaalv.toDouble();
    loss = (rawLoss <= -1) ? 0 : rawLoss;
  } else if ([5, 20, 6, 29, 30, 12, 13, 27, 28, 2, 1]
          .contains(FFAppState().cmbStatus) &&
      FFAppState().cmbStatus2 == 0) {
    final aalv = (totLoanValue - totilv).round();
    final totaalv = aalv + totialv;

    loss = (audit.pledgeBalance > totaalv)
        ? (audit.pledgeBalance - totaalv).toDouble()
        : 0.0;
  } else {
    loss = 0.0;
  }
  return loss;
}

bool _isStonePurity_Deductible() {
  final dt = DateTime(2013, 2, 8);
  final dt1 = DateTime(2012, 3, 27);
  return audit.pledgeDate.isAfter(dt) || audit.pledgeDate.isBefore(dt1);
}

Future<String> _stonePurityDeduction(
    String spurity, String grwt, String stwt, String itemname) async {
  double ded = double.tryParse(spurity) ?? 0.0;
  double g = double.tryParse(grwt) ?? 0.0;
  double s = double.tryParse(stwt) ?? 0.0;
  if (g <= 0) return "0";
  double net = g - s;
  double stone = (s / g) * 100;

  if (stone > 85 && itemname.toUpperCase() == "RAJPUTI") {
    _showMessage(
        "Rajaputi Item with stone weight greater than 85% of gross weight is not allowed. Ref : Circular :
5459");
  } else if (stone > 60 && itemname.toUpperCase() != "RAJPUTI") {
    _showMessage(
        "Items with stone weight greater than 50% of gross weight is not allowed. Ref : Circular : 4304");
  }

  double totalDed;
  double? apiDed =
      await ApiService().fetchStoneDeduction(audit.pledgeDate, stone);
  if (apiDed != null) {
```

```
      double extraDed = apiDed;
      if (stone > 60 && stone <= 85 && itemname.toUpperCase() == "RAJPUTI") {
        extraDed = 5.0;
      }
      totalDed = ded + extraDed;
    } else {
      totalDed = 0.0;
    }

    double sntwt = net - (net * totalDed / 100);
    double roundedSntwt = double.parse(sntwt.toStringAsFixed(2));

    return roundedSntwt < 0 ? "0" : roundedSntwt.toString();
  }

  Future<String> _alcStonePurityDeduction(
      String ntwt, String grwt, String stwt, String itemname) async {
    double g = double.tryParse(grwt) ?? 0.0;
    double s = double.tryParse(stwt) ?? 0.0;
    if (g <= 0) return ntwt;
    double net = g - s;
    double stone = (s / g) * 100;

    if (stone > 85 && itemname.toUpperCase() == "RAJPUTI") {
      _showMessage(
          "Rajaputi Item with stone weight greater than 85% of gross weight is not allowed. Ref : Circular :
5459");
    } else if (stone > 60 && itemname.toUpperCase() != "RAJPUTI") {
      _showMessage(
          "Items with stone weight greater than 50% of gross weight is not allowed. Ref : Circular : 4304");
    }

    double ded = 0.0;
    double? apiDed =
        await ApiService().fetchStoneDeduction(audit.pledgeDate, stone);
    if (apiDed != null) {
      if (stone > 60 && stone <= 85 && itemname.toUpperCase() == "RAJPUTI") {
        ded = 5.0;
      } else {
        ded = apiDed;
      }
    }

    double deductionAmount = double.parse((net * ded / 100).toStringAsFixed(2));
    double currentNtwt = double.tryParse(ntwt) ?? 0.0;
    double sntwt =
        double.parse((currentNtwt - deductionAmount).toStringAsFixed(2));

    return sntwt < 0 ? "0" : sntwt.toString();
  }

  Widget _cellNum({
    required double value,
    required ValueChanged<double> onChanged,
    required VoidCallback onSubmitted,
  }) {
    return _NumberCell(
        value: value, onChanged: onChanged, onSubmitted: onSubmitted);
  }

  Widget _cellTxt({
    required String value,
    required ValueChanged<String> onChanged,
    required VoidCallback onSubmitted,
  }) {
    return _TextCell(
        value: value, onChanged: onChanged, onSubmitted: onSubmitted);
  }

  @override
  Widget build(BuildContext context) {
    final autoLossStatuses = [
      -1, 0, 3, 7, 15, 16, 17, 18, 19, 21, 24, 25, 26, 32
    ];
    final isGridMode = !autoLossStatuses.contains(FFAppState().cmbStatus);

    String gwLabel = 'Gross Wt';
    String purityLabel = 'Purity';
    String estLossLabel = 'Purity Deduction in %';
    bool showPurity = true;
    bool lossreadOnly = true;

    if (FFAppState().cmbStatus == 2) gwLabel = 'Spurious G.W';
    if ([5, 6, 14].contains(FFAppState().cmbStatus))
```

```
    purityLabel = 'Estimated Purity';
  if (FFAppState().cmbStatus == 10) estLossLabel = 'Deduction in LoanAmount';

  if ([23, 9, 10, 4].contains(FFAppState().cmbStatus)) {
    showPurity = false;
  }

  if (_isLoading)
    return const Scaffold(
        backgroundColor: Color(0xFFF5F7FA),
        body: Center(child: CircularProgressIndicator()));

  return Scaffold(
    backgroundColor: const Color(0xFFF5F7FA),
    body: SingleChildScrollView(
      child: Column(
        children: [
          Padding(
            padding: const EdgeInsets.all(16.0),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.stretch,
              children: [
                Card(
                  elevation: 4,
                  shadowColor: Colors.black12,
                  color: Colors.white,
                  shape: RoundedRectangleBorder(
                      borderRadius: BorderRadius.circular(16)),
                  child: Padding(
                    padding: const EdgeInsets.all(20.0),
                    child: Column(
                      crossAxisAlignment: CrossAxisAlignment.start,
                      children: [
                        Text('Audit Status',
                            style: Theme.of(context).textTheme.titleMedium),
                        const SizedBox(height: 12),
                        Row(
                          children: [
                            Expanded(
                              child: DropdownButtonFormField<int>(
                                decoration: const InputDecoration(
                                  labelText: 'First irregularity',
                                  filled: true,
                                  fillColor: Colors.white,
                                  border: OutlineInputBorder(),
                                ),
                                value: _statusOptions
                                        .containsKey(FFAppState().cmbStatus)
                                    ? FFAppState().cmbStatus
                                    : null,
                                items: _statusOptions.entries
                                    .map((e) => DropdownMenuItem(
                                        value: e.key,
                                        child: Text(e.value,
                                            overflow: TextOverflow.ellipsis)))
                                    .toList(),
                                onChanged:
                                    isEnabled ? _handleStatusChange : null,
                              ),
                            ),
                            const SizedBox(width: 16),
                            Expanded(
                              child: Column(
                                children: [
                                  Container(
                                    padding: EdgeInsets.all(8),
                                    decoration: BoxDecoration(
                                      color: Colors.white,
                                      borderRadius: BorderRadius.circular(8),
                                      border: Border.all(
                                        color: Colors.black,
                                      ),
                                    ),
                                    width: double.infinity,
                                    height: 48,
                                    child: Center(
                                      child: Text(
                                        'First irregularity: ${((FFAppState().cmbStatus != 0) ?
_statusOptions[FFAppState().cmbStatus] ?? '' : '')}',
                                        style: Theme.of(context)
                                            .textTheme
                                            .bodyMedium!
                                            .copyWith(
                                                fontWeight: FontWeight.w500,
```

```
                        color: Colors.red),
                  ),
                ),
              )
            ],
          ),
        ),
        const SizedBox(width: 16),
        Expanded(
          child: DropdownButtonFormField<int>(
            decoration: const InputDecoration(
              labelText: 'Second irregularity',
              filled: true,
              fillColor: Colors.white,
              border: OutlineInputBorder(),
            ),
            value: _statusOptions
                    .containsKey(FFAppState().cmbStatus2)
                ? FFAppState().cmbStatus2
                : null,
            items: _statusOptions.entries
                .map((e) => DropdownMenuItem(
                    value: e.key,
                    child: Text(e.value,
                        overflow: TextOverflow.ellipsis)))
                .toList(),
            onChanged: isEnabled
                ? (v) {
                    setState(() {
                      FFAppState().cmbStatus2 =
                          v ?? FFAppState().cmbStatus2;
                      if (isGridMode)
                        _recalculateTotals();
                    });
                  }
                : null,
          ),
        ),
      ],
    ),
    if (FFAppState().cmbStatus == 3) ...[
      const SizedBox(height: 16),
      Container(
        width: 400,
        child: TextFormField(
          readOnly: lossreadOnly,
          decoration: InputDecoration(
            border: OutlineInputBorder(),
            hintText: 'Loss Amount: ${FFAppState().loss}',
          ),
          keyboardType:
              const TextInputType.numberWithOptions(
                  decimal: true),
          onChanged: (v) {
            setState(() {
              FFAppState().update(() {
                FFAppState().txtLoss = double.tryParse(v) ?? 0.0;
              });
            });
          },
        ),
      ),
    ],
    if (FFAppState().cmbStatus == 32) ...[
      const SizedBox(height: 16),
      Container(
        width: 400,
        child: TextFormField(
          decoration: InputDecoration(
            hintText: audit.tareWeight == 0
                ? 'Enter tare weight'
                : audit.tareWeight.toString(),
            border: OutlineInputBorder(),
          ),
          initialValue: audit.tareWeight == 0
              ? ''
              : audit.tareWeight.toString(),
          keyboardType:
              const TextInputType.numberWithOptions(
                  decimal: true),
          onChanged: (v) {
            setState(() {
              audit.tareWeight = double.tryParse(v) ?? 0.0;
              FFAppState().update(() {
```

```
                        FFAppState().newtareweight = v;
                      });
                    });
                  },
                ),
              ),
            ],
          ],
        ),
      ),
    ),

    if (isGridMode)
      Card(
        elevation: 8,
        shadowColor: const Color(0xFF667eea).withOpacity(0.4),
        color: Colors.white,
        shape: RoundedRectangleBorder(
            borderRadius: BorderRadius.circular(20)),
        child: Padding(
          padding: const EdgeInsets.all(24.0),
          child: Column(
            children: [
              Row(
                mainAxisAlignment: MainAxisAlignment.spaceBetween,
                children: [
                  _SummaryItem(
                      label: 'Total GW',
                      value: FFAppState()
                          .totalGwVar
                          .toStringAsFixed(3)),
                  _SummaryItem(
                      label: 'Total Stone',
                      value: FFAppState()
                          .totalStoneVar
                          .toStringAsFixed(3)),
                  _SummaryItem(
                      label: 'Total Net',
                      value: FFAppState()
                          .totalNetWtVar
                          .toStringAsFixed(3),
                      isBold: true),
                ],
              ),
              const Divider(height: 32),
              Row(
                mainAxisAlignment: MainAxisAlignment.center,
                children: [
                  Text('Estimated Loss: ',
                      style: Theme.of(context)
                          .textTheme
                          .titleMedium),
                  Text(
                    FFAppState().txtLoss.toStringAsFixed(2),
                    style: Theme.of(context)
                        .textTheme
                        .headlineSmall
                        ?.copyWith(
                          color: (double.tryParse(FFAppState()
                                          .txtLoss
                                          .toStringAsFixed(2)) ??
                                      0.0) >
                                  0
                              ? Colors.black
                              : Colors.black,
                          fontWeight: FontWeight.bold,
                        ),
                  ),
                ],
              ),
            ],
          ),
        ),
      ),
  ],
),
),
if (isGridMode) ...[
  Card(
    margin: const EdgeInsets.symmetric(horizontal: 16),
    elevation: 8,
    shadowColor: const Color(0xFF667eea).withOpacity(0.25),
    clipBehavior: Clip.antiAlias,
    color: Colors.white,
```

```
shape: RoundedRectangleBorder(
    borderRadius: BorderRadius.circular(16)),
child: SingleChildScrollView(
  scrollDirection: Axis.horizontal,
  child: DataTable(
    columnSpacing: 20,
    headingRowColor:
        MaterialStateProperty.all(const Color(0xFF667eea)),
    headingTextStyle: const TextStyle(
        fontWeight: FontWeight.bold, color: Colors.white),
    columns: [
      const DataColumn(
          label: Text('Item Name',
              style: const TextStyle(
                  fontWeight: FontWeight.bold, fontSize: 11))),
      const DataColumn(
          label: Text('Select',
              style: const TextStyle(
                  fontWeight: FontWeight.bold, fontSize: 11))),
      const DataColumn(
          label: Text('No of Pieces',
              style: const TextStyle(
                  fontWeight: FontWeight.bold, fontSize: 11))),
      DataColumn(label: Text(gwLabel)),
      const DataColumn(
          label: Text('Net Wt',
              style: const TextStyle(
                  fontWeight: FontWeight.bold, fontSize: 11))),
      const DataColumn(
          label: Text('Stone Wt',
              style: const TextStyle(
                  fontWeight: FontWeight.bold, fontSize: 11))),
      const DataColumn(
          label: Text('CurrentPurity',
              style: const TextStyle(
                  fontWeight: FontWeight.bold, fontSize: 11))),
      if (showPurity)
        DataColumn(
            label: Text('Item Type',
                style: const TextStyle(
                    fontWeight: FontWeight.bold,
                    fontSize: 11))),
      if (showPurity)
        DataColumn(
            label: Text(purityLabel,
                style: const TextStyle(
                    fontWeight: FontWeight.bold,
                    fontSize: 11))),
      if (showPurity)
        DataColumn(
            label: Text(estLossLabel,
                style: const TextStyle(
                    fontWeight: FontWeight.bold,
                    fontSize: 11))),
    ],
    rows: List.generate(FFAppState().DtviewIrr.length, (i) {
      final r = FFAppState().DtviewIrr[i];
      final purityOptions = _purityOptionsList.length > i
          ? _purityOptionsList[i]
          : <String>[];
      return DataRow(
        cells: [
          DataCell(SizedBox(
              width: 120,
              child: Text(r.colItem ?? '',
                  style: const TextStyle(
                      fontWeight: FontWeight.w500,
                      fontSize: 11)))),
          DataCell(Checkbox(
            value: r.colStatus,
            onChanged: (v) {
              setState(() => r.colStatus = v ?? false);
              _dtviewIrrCellEndEdit(i, 1);
            },
          )),
          DataCell(_cellNum(
            value: double.tryParse(r.colItemcount ?? '0') ?? 0.0,
            onChanged: (v) {
              r.colItemcount = v.toInt().toString();
              _dtviewIrrCellEndEdit(i, 2);
            },
            onSubmitted: () => _dtviewIrrCellEndEdit(i, 2),
          )),
          DataCell(_cellNum(
```

```
  value: double.tryParse(r.colGw ?? '0') ?? 0.0,
  onChanged: (v) {
    r.colGw = v.toString();
    _dtviewIrrCellEndEdit(i, 3);
  },
  onSubmitted: () => _dtviewIrrCellEndEdit(i, 3),
)),
DataCell(Text(
    (double.tryParse(r.colNetwt ?? '0') ?? 0.0)
        .toStringAsFixed(3),
    style: const TextStyle(
        fontWeight: FontWeight.bold, fontSize: 11))),
DataCell(_cellNum(
  value: double.tryParse(r.colStwt ?? '0') ?? 0.0,
  onChanged: (v) {
    r.colStwt = v.toString();
    _dtviewIrrCellEndEdit(i, 5);
  },
  onSubmitted: () => _dtviewIrrCellEndEdit(i, 5),
)),
DataCell(SizedBox(
    width: 120,
    child: Text(r.colCurpurity ?? '',
        style: const TextStyle(
            fontWeight: FontWeight.w500,
            fontSize: 11)))),
if (showPurity)
  DataCell(SizedBox(
    width: 160,
    child: _itemTypes.isNotEmpty
        ? DropdownButtonFormField<int>(
            hint: const Text('Select Type',
                style: TextStyle(fontSize: 11)),
            value: _itemTypes.containsKey(r.colTypid)
                ? r.colTypid
                : null,
            isExpanded: true,
            decoration: const InputDecoration(
                border: InputBorder.none,
                contentPadding: EdgeInsets.zero),
            items: _itemTypes.entries
                .map((e) => DropdownMenuItem(
                      value: e.key,
                      child: Text(e.value,
                          overflow:
                              TextOverflow.ellipsis,
                          style: const TextStyle(
                              fontSize: 11)),
                    ))
                .toList(),
            onChanged: (v) {
              if (v != null) _handleTypeChange(i, v);
            },
          )
        : Text(_itemTypes[r.colTypid] ?? ''),
  )),
if (showPurity)
  DataCell(SizedBox(
    width: 160,
    child: purityOptions.isNotEmpty
        ? DropdownButtonFormField<String>(
            hint: const Text('Select Purity',
                style: TextStyle(fontSize: 11)),
            value: purityOptions.contains(r.colPurity)
                ? r.colPurity
                : null,
            isExpanded: true,
            decoration: const InputDecoration(
                border: InputBorder.none,
                contentPadding: EdgeInsets.zero),
            items: purityOptions
                .map((e) => DropdownMenuItem(
                      value: e,
                      child: Text(e.split('~').first,
                          overflow:
                              TextOverflow.ellipsis,
                          style: const TextStyle(
                              fontSize: 11)),
                    ))
                .toList(),
            onChanged: (v) {
              if (v != null) {
                r.colPurity = v;
                _dtviewIrrCellEndEdit(i, 8);
```

```
                                    }
                                  },
                                )
                              : _cellTxt(
                                  value: r.colPurity ?? '',
                                  onChanged: (v) {
                                    r.colPurity = v;
                                    _dtviewIrrCellEndEdit(i, 8);
                                  },
                                  onSubmitted: () =>
                                      _dtviewIrrCellEndEdit(i, 8),
                                ),
                          )),
                      if (showPurity)
                        DataCell(Text(
                            (double.tryParse(r.colEstimat ?? '0') ?? 0.0)
                                .toStringAsFixed(0),
                            style: const TextStyle(
                                fontWeight: FontWeight.bold,
                                fontSize: 11))),
                    ],
                  );
                }),
              ),
            ),
          ),
        ] else
          Center(
            child: Column(
              children: [
                const SizedBox(height: 40),
              ],
            ),
          ),
        const SizedBox(height: 80),
      ],
    ),
  ),
);
  }
}

class _InfoCard extends StatelessWidget {
  final String title;
  final String value;
  final IconData icon;
  const _InfoCard(
      {required this.title, required this.value, required this.icon});

  @override
  Widget build(BuildContext context) {
    return Card(
      elevation: 0,
      color: Colors.white,
      child: Padding(
        padding: const EdgeInsets.symmetric(vertical: 12, horizontal: 8),
        child: Column(
          children: [
            Icon(icon, size: 20, color: Theme.of(context).colorScheme.primary),
            const SizedBox(height: 4),
            Text(title, style: Theme.of(context).textTheme.labelSmall),
            Text(value,
                style: Theme.of(context)
                    .textTheme
                    .titleMedium
                    ?.copyWith(fontWeight: FontWeight.bold)),
          ],
        ),
      ),
    );
  }
}

class _SummaryItem extends StatelessWidget {
  final String label;
  final String value;
  final bool isBold;
  final Color? textColor;
  const _SummaryItem(
      {required this.label,
      required this.value,
      this.isBold = false,
      this.textColor});
```

```
  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        Text(label,
            style: Theme.of(context)
                .textTheme
                .bodySmall
                ?.copyWith(color: textColor?.withOpacity(0.8))),
        Text(value,
            style: Theme.of(context).textTheme.titleMedium?.copyWith(
                fontWeight: isBold ? FontWeight.bold : FontWeight.normal,
                color: textColor,
                fontSize: 18)),
      ],
    );
  }
}

// ================== MODELS / HANDLERS ==================

class AuditState {
  double lendRate;
  int pledgeBalance;
  double totalNetWtDb;
  String scheme;
  DateTime pledgeDate;
  double marketRate;
  double actweight;
  double netweight;

  int? spotRectificationStatus;
  String spotRemarks = '';
  DateTime rectificationDate = DateTime.now();
  int kycStatus = 0;
  int invStatus = 0;
  bool isGoldBar = false;
  bool isGoldCoin = false;
  String remarks = '';
  String packetIntact = '';
  String auditorNotice = '';
  String prevAuditor = '';
  String multipleIds = '';
  int recoveryProspect = -1;
  String firNo = '';
  String blobPath = '';
  double tareWeight = 0.0;

  AuditState({
    required this.lendRate,
    required this.pledgeBalance,
    required this.totalNetWtDb,
    required this.scheme,
    required this.pledgeDate,
    required this.actweight,
    required this.netweight,
    this.marketRate = 0.0,
  });
}

typedef ShowMessage = void Function(String message);

class AlcInput {
  final String itemName;
  final double deduRate;
  final int dbItemCount;
  final int itemId;
  final int brid;
  final String pledgeNo;
  final double cgwt;
  final double cstwt;
  final int statusMain;
  final double cntwt;
  final String scheme;
  final DateTime pledgeDate;
  final int statusSub;
  final int typeId;
  final String curPurity;
  final String itemType;
  final double lendRate;
  AlcInput({
    required this.itemName,
    required this.deduRate,
    required this.dbItemCount,
```

```
    required this.itemId,
    required this.brid,
    required this.pledgeNo,
    required this.cgwt,
    required this.cstwt,
    required this.statusMain,
    required this.cntwt,
    required this.scheme,
    required this.pledgeDate,
    required this.statusSub,
    required this.typeId,
    required this.curPurity,
    required this.itemType,
    required this.lendRate,
  });
}

class AlcResult {
  final double tGrwt, tStwt, tNtwt;
  final String subrecord;
  AlcResult(
      {required this.tGrwt,
      required this.tStwt,
      required this.tNtwt,
      required this.subrecord});
}

typedef OpenAlcForm = Future<AlcResult?> Function(AlcInput input);

double parsePurityPercent(String purityText) {
  if (purityText.isEmpty) return 0.0;
  final parts = purityText.split('~');
  if (parts.length > 1) {
    final maybe = double.tryParse(parts[1].trim());
    if (maybe != null) return maybe;
  }
  final first = parts.first.trim();
  if (first.startsWith('Purity Above 85')) return 85.0;
  final reg = RegExp(r'(\d+(\.\d+)?)');
  final m = reg.firstMatch(first);
  return m != null ? double.tryParse(m.group(1)!) ?? 0.0 : 0.0;
}

bool skipDeductionByScheme(String scheme) => false;

bool isStonePurityDeductible(DateTime pledgeDate) {
  final start = DateTime(2013, 2, 8), end = DateTime(2012, 3, 27);
  return pledgeDate.isAfter(start) || pledgeDate.isBefore(end);
}

Future<double> stonePurityDeduction({
  required double spurityPercent,
  required double grwt,
  required double stwt,
  required String itemName,
  required DateTime pledgeDate,
  required ShowMessage showMessage,
  Future<double?> Function(double stonePct, DateTime pledgeDate)?
      stoneDeductionLookup,
}) {
  final g = grwt;
  if (g <= 0) return Future.value(0.0);
  final s = stwt;
  return Future(() async {
    final stonePct = (s / g) * 100.0;
    if (stonePct > 85 && itemName.toUpperCase() == 'RAJPUTI') {
      showMessage('Rajaputi item stone >85% not allowed (Circular 5459)');
    } else if (stonePct > 60 && itemName.toUpperCase() != 'RAJPUTI') {
      showMessage('Items stone >50% not allowed (Circular 4304)');
    }
    double? dedExtra;
    if (stoneDeductionLookup != null)
      dedExtra = await stoneDeductionLookup(stonePct, pledgeDate);

    double totalDed = 0.0;
    if (dedExtra != null) {
      double extra = dedExtra;
      if (stonePct > 60 &&
          stonePct <= 85 &&
          itemName.toUpperCase() == 'RAJPUTI') extra = 5.0;
      totalDed = max(0.0, spurityPercent + extra);
    } else {
      totalDed = 0.0;
    }
```

```
    final netGold = max(0.0, g - s);
    final sntwt = (netGold - (netGold * totalDed / 100.0));
    return sntwt <= 0 ? 0.0 : double.parse(sntwt.toStringAsFixed(2));
  });
}

Future<double> alcStonePurityDeduction({
  required double ntwt,
  required double grwt,
  required double stwt,
  required String itemName,
  required DateTime pledgeDate,
  required ShowMessage showMessage,
  Future<double?> Function(double stonePct, DateTime pledgeDate)?
      stoneDeductionLookup,
}) {
  final g = grwt;
  if (g <= 0) return Future.value(0.0);
  final s = stwt;
  return Future(() async {
    final stonePct = (s / g) * 100.0;
    if (stonePct > 85 && itemName.toUpperCase() == 'RAJPUTI') {
      showMessage('Rajaputi item stone >85% not allowed (Circular 5459)');
    } else if (stonePct > 60 && itemName.toUpperCase() != 'RAJPUTI') {
      showMessage('Items stone >50% not allowed (Circular 4304)');
    }
    double? dedExtra;
    if (stoneDeductionLookup != null)
      dedExtra = await stoneDeductionLookup(stonePct, pledgeDate);

    double dedVal = 0.0;
    if (dedExtra != null) {
      if (stonePct > 60 &&
          stonePct <= 85 &&
          itemName.toUpperCase() == 'RAJPUTI') {
        dedVal = 5.0;
      } else {
        dedVal = dedExtra;
      }
    } else {
      dedVal = 0.0;
    }

    final netGold = max(0.0, g - s);
    final stoneDed = netGold * (max(0.0, dedVal) / 100.0);
    final val = ntwt - double.parse(stoneDed.toStringAsFixed(2));
    return val <= 0 ? 0.0 : double.parse(val.toStringAsFixed(2));
  });
}

class _NumberCell extends StatefulWidget {
  final double value;
  final ValueChanged<double> onChanged;
  final VoidCallback onSubmitted;
  const _NumberCell(
      {required this.value,
      required this.onChanged,
      required this.onSubmitted});
  @override
  State<_NumberCell> createState() => _NumberCellState();
}

class _NumberCellState extends State<_NumberCell> {
  late TextEditingController _ctrl;
  @override
  void initState() {
    super.initState();
    _ctrl = TextEditingController(
        text: widget.value == 0 ? '' : widget.value.toString());
  }

  @override
  void didUpdateWidget(_NumberCell oldWidget) {
    super.didUpdateWidget(oldWidget);
    if (widget.value != oldWidget.value) {
      final currentVal = double.tryParse(_ctrl.text) ?? 0.0;
      if ((currentVal - widget.value).abs() > 0.0001) {
        _ctrl.text = widget.value == 0 ? '' : widget.value.toString();
        _ctrl.selection = TextSelection.collapsed(offset: _ctrl.text.length);
      }
    }
  }
```

```
  @override
  void dispose() {
    _ctrl.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return SizedBox(
      width: 100,
      child: TextField(
        controller: _ctrl,
        keyboardType: TextInputType.number,
        onChanged: (v) => widget.onChanged(double.tryParse(v.trim()) ?? 0.0),
        onSubmitted: (_) => widget.onSubmitted(),
        decoration:
            const InputDecoration(isDense: true, border: OutlineInputBorder()),
      ),
    );
  }
}

class ApiService {
  static String baseUrl = FFDevEnvironmentValues().Baseurl +
      '/AuditInspectionApi/api/GetInspectionData';

  String token = FFAppState().Token;

  Future<List<dynamic>> _get(String flag, String params) async {
    final url = Uri.parse('$baseUrl/$flag/$params/1');
    try {
      final response = await http.get(
        url,
        headers: {'Authorization': 'Bearer $token'},
      );
      if (response.statusCode == 200) {
        final data = jsonDecode(response.body);
        if (data is Map && data.containsKey('result')) {
          return data['result'] as List<dynamic>;
        }
      } else {
        debugPrint('API Error ${response.statusCode}: ${response.body}');
      }
    } catch (e) {
      debugPrint('API Exception: $e');
    }
    return [];
  }

  Future<int> checkAuditMain(String user, int flag) async {
    final data = await _get('PROC_AUDIT_MAIN', '$flag*$user');
    if (data.isNotEmpty) {
      return int.tryParse(data[0]['COUNT(*)']?.toString() ?? '0') ?? 0;
    }
    return 0;
  }

  Future<List<Result1Struct>> fetchPledgeItems(
      String pledgeNo, String brid) async {
    final data = await _get(
        'IRREGULARITY_VIEW', '$pledgeNo*${FFAppState().cmbStatus}*$brid');
    return data.map((json) {
      String itemName = '';
      json.forEach((k, v) {
        if (k.toString().contains('ITEM_NAME')) itemName = v.toString().trim();
        if (k.toString().contains('RPAD(IM.ITEM_NAME,30)'))
          itemName = v.toString().trim();
      });
      return Result1Struct(
        colItem: itemName,
        colItemid: int.tryParse(json['ITEM_ID']?.toString() ?? '0') ?? 0,
        colCurpurity: json['DEDUCTION_NAME']?.toString() ?? '',
        colDeducrate: int.tryParse(
                json['DEDUCTION_RATE']?.toString().split('~').last ?? '0') ??
            0,
        colAutoloss: '0',
        colDbitemcount:
            int.tryParse(json['ITEM_COUNT']?.toString() ?? '0') ?? 0,
        colTypid: int.tryParse(json['TYPE_ID']?.toString() ?? '0') ?? 0,
        colCgwt: double.tryParse(json['ACT_WT']?.toString() ?? '0') ?? 0.0,
        colCstwt: int.tryParse(json['STONE_WT']?.toString() ?? '0') ?? 0,
        colCntwt: double.tryParse(json['NET_WT']?.toString() ?? '0') ?? 0.0,
        colPurity: json['DEDUCTION_NAME']?.toString() ?? '',
        colStatus: false,
```

```
      colItemcount: '0',
      colGw: '0',
      colStwt: '0',
      colNetwt: '0',
      colEstimat: '0',
      colCirNo: '',
      colSubrecord: '',
      colItemtype: 0,
    );
  }).toList();
}

Future<double?> fetchStoneDeduction(DateTime date, double stonePct) async {
  const months = [
    'Jan',
    'Feb',
    'Mar',
    'Apr',
    'May',
    'Jun',
    'Jul',
    'Aug',
    'Sep',
    'Oct',
    'Nov',
    'Dec'
  ];
  final dateStr =
      "${date.day.toString().padLeft(2, '0')}-${months[date.month - 1]}-${date.year}";
  final data = await _get(
      'PLP_AUDIT_SELECT', 'STONEDEDUCTION*0*0*$dateStr*$stonePct*0');
  return data.isNotEmpty
      ? (double.tryParse(data[0]['DED_PERC']?.toString() ?? '0'))
      : null;
}

Future<double> fetchMarketRate() async {
  final data = await _get('PLP_AUDIT_SELECT', 'MARKETRATE*0*0*0*0*0');
  return data.isNotEmpty
      ? (double.tryParse(data[0]['PARMTR_VALUE'].toString()) ?? 0.0)
      : 0.0;
}

Future<double> fetchLendRate(String pledgeNo) async {
  final data = await _get('PLP_AUDIT_SELECT', 'LENDRATE*$pledgeNo*0*0*0*0');
  return data.isNotEmpty
      ? (double.tryParse(data[0]['LND_RATE'].toString()) ?? 0.0)
      : 0.0;
}

Future<int> fetchPledgeBalance(String pledgeNo) async {
  final data = await _get('LOAD_PLEDGE_BALANCE', pledgeNo);
  if (data.isEmpty) return 0;
  final val = data[0]['PLEDGE_BAL'];
  return val != null ? (double.tryParse(val.toString())?.toInt() ?? 0) : 0;
}

Future<List<String>> fetchPurityParams(String pledgeNo, String brid,
    int itemType, String scheme, DateTime date) async {
  const months = [
    'Jan',
    'Feb',
    'Mar',
    'Apr',
    'May',
    'Jun',
    'Jul',
    'Aug',
    'Sep',
    'Oct',
    'Nov',
    'Dec'
  ];
  final dateStr =
      "${date.day.toString().padLeft(2, '0')}-${months[date.month - 1]}-${date.year}";
  final params = 'PURITYPARAMS*$pledgeNo*$brid*$itemType*$scheme*$dateStr';
  final data = await _get('PLP_AUDIT_SELECT', params);
  return data.map((e) => e['DEDUCTION_RATE'].toString()).toList();
}

Future<Map<int, String>> fetchItemTypes() async {
  final data = await _get('PLP_AUDIT_SELECT', 'ITEMTYPES*0*0*0*0*0');
  final options = <int, String>{};
  for (final item in data) {
```

```
      final id = item['TYPE_ID'];
      final name = item['TYPE_NAME'];
      if (id != null && name != null) {
        options[int.parse(id.toString())] = name.toString();
      }
    }
    return options;
  }

  Future<Map<int, String>> fetchStatusOptions(String brid, int stflg) async {
    final data = await _get('STATEID_CHECK', '$brid*$stflg');
    final options = <int, String>{};
    for (final item in data) {
      var id;
      var name;

      if (item is Map) {
        for (var key in item.keys) {
          if (key == '0' || key == 'STATUS_ID') id = item[key];
          if (key == "'-------SELECT---------'" || key == 'STATUS_NAME')
            name = item[key];
        }
        if (id == null && item.values.isNotEmpty) {
          final vals = item.values.toList();
          if (vals.isNotEmpty) id = vals[0];
          if (vals.length > 1) name = vals[1];
        }
      }

      if (id != null && name != null) {
        final parsedId = double.tryParse(id.toString())?.toInt();
        if (parsedId != null) {
          options[parsedId] = name.toString();
        }
      }
    }
    return options;
  }

  Future<Map<String, dynamic>?> fetchInventoryDetails(
      String brid, String pledgeNo) async {
    final data = await _get('LOAD_INV_DETAILS', '$brid*$pledgeNo');
    if (data.isNotEmpty) {
      return data[0];
    }
    return null;
  }
}

class _TextCell extends StatefulWidget {
  final String value;
  final ValueChanged<String> onChanged;
  final VoidCallback onSubmitted;
  const _TextCell(
      {required this.value,
      required this.onChanged,
      required this.onSubmitted});
  @override
  State<_TextCell> createState() => _TextCellState();
}

class _TextCellState extends State<_TextCell> {
  late TextEditingController _ctrl;
  @override
  void initState() {
    super.initState();
    _ctrl = TextEditingController(text: widget.value);
  }

  @override
  void didUpdateWidget(_TextCell oldWidget) {
    super.didUpdateWidget(oldWidget);
    if (widget.value != oldWidget.value && widget.value != _ctrl.text) {
      _ctrl.text = widget.value;
      _ctrl.selection = TextSelection.collapsed(offset: _ctrl.text.length);
    }
  }

  @override
  void dispose() {
    _ctrl.dispose();
    super.dispose();
  }
```

```
  @override
  Widget build(BuildContext context) {
    return SizedBox(
      width: 160,
      child: TextField(
        controller: _ctrl,
        onChanged: widget.onChanged,
        onSubmitted: (_) => widget.onSubmitted(),
        decoration:
            const InputDecoration(isDense: true, border: OutlineInputBorder()),
      ),
    );
  }
}
```